

Software requirements and installation

Software requirements and installation

Spatially explicit coalescent simulations of glacial cycles in the Australian Alps

September 2026

What this document covers

This is the installation guide for the simulation code that accompanies the manuscript. It lists the software the simulations were run with, explains how to install it on a clean machine, and gives a single command that checks the whole chain end to end before you commit to a long run.

The code has two layers. The coalescent simulation itself is Python and uses **msprime**. Everything around it — the study design, the batch runners, the population-genetic summaries and the figures — is **R**, and reaches into Python through the **reticulate** package. You therefore need both, and they need to be able to see each other.

Layer	Does
Python (python/glacial_sim.py)	Builds the landscape, the climate cycles and the msprime demography; simulates tree sequences; computes per-deme summary statistics
R (R/* .R)	Design matrix, serial and parallel batch runners, genlight conversion, all-pairs Hudson Fst, figures

Software used

Python

Package	Minimum version	Purpose
Python	3.9 (3.11 recommended)	interpreter
msprime	1.4	coalescent simulation with a time-varying structured demography
tskit	0.5	tree-sequence statistics (installed with msprime)
numpy	2.0	array handling (numpy 1.x only with msprime < 1.4)

No other Python packages are needed. msprime pulls in tskit automatically.

R

Package	Purpose
R	4.2 or newer
reticulate	bridge from R to the Python engine; 1.38 or newer (see below)
adegenet	genlight objects for the simulated genotypes
dplyr, tibble, tidyr	table handling in the extraction step
ggplot2, patchwork	Figures 5 and 6
mgcv	shrinkage splines for the isolation-by-distance correction
scales	Figure 1
parallel	batch runner (ships with R)

dartRverse is **not** required. Earlier versions of the scripts loaded it, but only adegenet functionality is actually used, and adegenet alone is a much lighter dependency for anyone who just wants to re-run the simulations.

Figure 1 additionally uses scales , and Figure 2 uses base graphics only.

`dartRverse` is needed by exactly one optional function, `plot_fst_present_scenarios()` in `R/09_fig_supplementary.R`. Every published figure can be reproduced without it.

The one version pairing that matters

`reticulate` has to hand numpy arrays back to R. Support for numpy 2 arrived in `reticulate` 1.38, and `msprime` 1.4 onwards requires numpy 2. So use either

- `reticulate` $\geq$ 1.38 with `msprime` $\geq$ 1.4 and numpy $\geq$ 2 (recommended), or
- an older `reticulate` with numpy $<$ 2 and `msprime` $<$ 1.4.

Mixing an old `reticulate` with numpy 2 leaves `reticulate` unable to convert the arrays, and the simulation fails on its first call. `init_python()` checks for this explicitly and stops with an explanation rather than letting it surface later as an obscure coercion error.

Recording the exact versions of a run

Both batch runners write a `session_info.txt` next to the results, holding the full R `sessionInfo()` plus the Python, `msprime`, `tskit` and `numpy` versions in use. Every set of results is therefore self-documenting. To print the same information at any time:

```
source("R/00_setup.R")
init_python()          # reports python / msprime / numpy
sessionInfo()
```

Installation

Step 1 — Python and `msprime`, via `conda`

`msprime` is easiest to install with `conda` (Miniconda or Miniforge). If you do not have `conda`, install Miniforge from <https://conda-forge.org/download/>.

From the repository root:

```
conda env create -f environment.yml
conda activate msprime-env
python -c "import msprime; print(msprime.__version__)"
```

`environment.yml` pins the channel to `conda-forge` and installs Python 3.11, `msprime`, `tskit` and `numpy` into an environment called `msprime-env`.

If you would rather not use `conda`, `pip install msprime` works on all three platforms and installs a wheel with no compilation required:

```
python3 -m venv ~/msprime-env
source ~/msprime-env/bin/activate    # Windows: ~\msprime-env\Scripts\activate
pip install msprime numpy
```

In that case point `reticulate` at the virtual environment with `reticulate::use_virtualenv()` instead of `use_condaenv()` in `R/00_setup.R`.

Step 2 — R packages

In R:

```
install.packages(c("reticulate", "adegenet", "dplyr", "tibble", "tidyr",
                  "ggplot2", "patchwork", "mgcv", "scales"))
```

Step 3 — tell R where Python lives

The code never hard-codes a path. It reads the environment variable `GLACIALSIM_CONDA_ENV`, falling back to the environment name `msprime-env`. Set it once, either in your shell or in `~/Renvirom`:

```
GLACIALSIM_CONDA_ENV=/home/yourname/miniconda3/envs/msprime-env
```

An environment *name* works too if `conda` is on your path:

```
GLACIALSIM_CONDA_ENV=msprime-env
```

You can also pass the path directly: `init_python("/path/to/env")`.

To find the path of an existing environment: `conda env list`.

Step 4 — check the installation

From the repository root:

```
Rscript scripts/00_check_install.R
```

This reports every package version, connects R to Python, and then runs a deliberately tiny simulation (a 5 x 5 grid, one glacial cycle, three sampling times) all the way through to a `genlight` object and a table of summary statistics. It takes well under a minute. If it completes, the full pipeline will run.

Running the simulations

```
Rscript scripts/01_run_simulations.R      # simulate; writes results/batch/*.rds
Rscript scripts/02_extract_summaries.R    # present-day Fst and diversity
Rscript scripts/03_make_figures.R         # Figures 1, 2, 5 and 6
```

Edit the small settings block at the top of `scripts/01_run_simulations.R` first: the conda environment, the output directory, the number of replicates and the number of cores.

What it costs

The published design is 3 landscapes x 30 replicates = 90 runs, and each run simulates both thermal ecologies across 100 sampling times. A single replicate takes on the order of one to a few hours on one core, so the whole design is an overnight job on 30 cores and a multi-week job on one. Set `N_REPS <- 1` for a first pass.

Memory is the other constraint. Each worker holds one tree sequence and one present-day genotype matrix at a time, which is a few hundred megabytes to a couple of gigabytes depending on how many demes are occupied. Budget roughly 2 GB per core and do not set `N_CORES` higher than your RAM allows.

The extraction step (`02_extract_summaries.R`) is memory-hungry rather than slow, because it reads back the stored genotype matrices; it deliberately holds only one thermal ecology in memory at a time.

Parallel execution

The parallel runner uses a PSOCK cluster rather than forking. That is deliberate: forked processes and an embedded Python interpreter do not coexist reliably, whereas a PSOCK worker starts a clean R session and its own Python interpreter. Each worker runs one replicate from start to finish and writes its own output file, so a batch can be interrupted and resumed — completed runs are detected and skipped.

On a cluster with a job scheduler, the simplest approach is to run `scripts/01_run_simulations.R` as a single multi-core job. There is nothing in the code that requires shared memory between workers.

Reproducibility

Every replicate is fully determined by its seed. The design matrix stores one seed per replicate, and within a replicate the three simulation stages use `seed`, `seed + 100` and `seed + 200`, so re-running a design reproduces the same tree sequences exactly.

The seeds used for the published runs are built into `make_design()` and are also saved as `design/published_design_matrix.csv`. Both thermal ecologies of a replicate share a seed and a climate history, so warm- and cold-adapted results within a replicate are a matched pair; the same seed is reused across the three landscape scenarios, so scenario contrasts are paired as well.

One caveat: coalescent simulators guarantee reproducibility for a *fixed* msprime version. Different msprime releases can change the internal ordering of random draws, so results reproduce exactly on the same version and only statistically on a different one. This is why the version is recorded in `session_info.txt`.

Platform notes

The simulations were run on Linux. macOS behaves identically. On Windows, everything works, but two things are worth knowing:

- give `GLACIALSIM_CONDA_ENV` the full path to the environment (for example `C:/Users/you/.conda/envs/msprime-env`), using forward slashes;
- the PSOCK cluster starts fresh R processes, so each worker re-imports Python; this is slower to start up on Windows than on Linux, but it does work.

Troubleshooting

Python not initialised - call `init_python()` first

`R/00_setup.R` has been sourced but `init_python()` has not been run, or it failed. Run it explicitly and read the version report it prints.

ModuleNotFoundError: No module named 'msprime'

reticulate has attached to the wrong interpreter. Check with `reticulate::py_config()`. This almost always means `GLACIALSIM_CONDA_ENV` points somewhere else, or that reticulate had already initialised Python before `use_condaenv()` was called — start a fresh R session and call `init_python()` before anything else touches Python.

reticulate cannot convert numpy arrays

the version pairing described above. Upgrade reticulate with `install.packages("reticulate")`, or pin `numpy < 2` and `msprime < 1.4` in the conda environment. Check with `reticulate::py_numpy_available()`, which must return `TRUE`.

Workers die with an out-of-memory error

reduce `N_CORES`. Each worker needs its own copy of the tree sequence.

Figures 5 and 6 fail on `unique(pw$grid_n)`

`results/pw_dv.rds` was produced by an older version of the extraction code that did not record the grid size. Re-run `scripts/02_extract_summaries.R`.